

PYTHON FOR QUANT TRADERS · PART I

Python Basics — เริ่มเขียนโค้ดบรรทัดแรก

บทที่ 6–10: Setup · Variables · Control Flow · Data Structures · NumPy/Pandas · Visualization

จบ Part นี้ → โหลดข้อมูลตลาด วิเคราะห์ และ plot กราฟได้จริง

🚫 อ่าน Part นี้ยังไ้

● **เขียนโค้ดครั้งแรก** — อ่านช้า ๆ ตามลำดับ ห้ามข้าม · บทที่ 6.3 "อ่านโค้ดให้ออกเสียง" คือหัวใจของทั้ง Part — ถ้าอ่านโค้ดเป็นประโยคภาษาคนได้ ที่เหลือจะง่ายขึ้นทั้งหมด · ในทุกบทให้หากล่อง 📦 **อ่านว่า:** ก่อนพยายามแกะโค้ดเอง · และพิมพ์ตามจริง ๆ อย่าแค่อ่าน — นิวจำได้เร็วกว่าตาจำ

● **เขียน Python เป็นแล้ว** — ข้ามบทที่ 6-8 ได้ แต่อย่าข้าม 6.1 (virtual environment) ถ้ายังไม่เคยใช้ · เริ่มจริงที่บทที่ 9 (NumPy/Pandas) และ 10 (Visualization) · ของสำหรับคุณ: กล่อง 📦 **แกะทีละชั้น** เรื่อง list comprehension, กับดัก pandas alignment ในบทที่ 9, และหัวข้อ 10.4 ที่ทำให้ fat tail กลายเป็นตัวเลขที่เอียงไม่ได้

🎯 สิ่งที่จะทำได้เมื่อจบ PART I

- โหลดข้อมูลราคาหุ้น/crypto จาก CSV เข้า Python
 - คำนวณ daily return, rolling mean, volatility ได้ใน 3-5 บรรทัด
 - กรองข้อมูล หาวันที่ผลตอบแทนเกิน threshold ได้
 - plot กราฟราคาและ return ได้ พร้อม label และ title
- ทุกอย่างนี้เป็นพื้นฐานของ quant analysis ทั้งหมดที่จะตามมา

บทที่ 6: Setup & Python Basics

แก่นของบทนี้

Python คือภาษาที่ "อ่านออกเหมือนภาษาอังกฤษ" — ไม่ต้องประกาศ type, ไม่ต้อง compile, รันได้ทันที
บทนี้ตั้ง environment และสร้างความคุ้นเคยกับ variable, type, และ operator ซึ่งเป็นอริฐกัอนแรกของ
ทุก program

6.1 ติดตั้ง Environment

ขั้นตอน 5 ขั้น (ทำครั้งเดียวต่อโปรเจกต์)

1. **Python 3.10+** — ดาวน์โหลดจาก python.org เลือก "Add to PATH" ตอนติดตั้ง
2. **VS Code** — ดาวน์โหลดจาก code.visualstudio.com → ติดตั้ง Extension "Python" ของ Microsoft
3. **สร้างโฟลเดอร์โปรเจกต์ + virtual environment** — สำคัญมาก อธิบายด้านล่าง
4. **ติดตั้งไลบรารี** ลงใน environment นั้น
5. **ทดสอบ** — สร้างไฟล์ test.py พิมพ์ `print("Hello Quant")` แล้วรัน (F5 หรือ `python test.py`)

ขั้นที่ 3–4: virtual environment — ขั้นที่คนข้ามแล้วเสียเวลาที่หลังมากที่สุด

คำสั่ง `pip install pandas` เฉย ๆ จะติดตั้งลง **Python กลางของเครื่อง** ซึ่งใช้ร่วมกันทุกโปรเจกต์ · ตอนมีโปรเจกต์
เดียวก็ดูไม่มีปัญหา แต่พอมีโปรเจกต์ที่สองที่ต้องการ pandas คนละเวอร์ชัน คุณจะติดกับดักที่แก้ยากมาก

```
# — macOS / Linux —
mkdir quant-project && cd quant-project
python3 -m venv .venv          # สร้างกล่องแยกชื่อ .venv
source .venv/bin/activate      # เข้าไปอยู่ในกล่อง

# — Windows (PowerShell) —
mkdir quant-project; cd quant-project
python -m venv .venv
.venv\Scripts\Activate.ps1

# หลัง activate: หน้า prompt จะมี (.venv) นำหน้า
(.venv) $ pip install numpy pandas matplotlib scipy statsmodels

# จดรายการไลบรารีลงไฟล์ เพื่อให้เครื่องอื่น (หรือตัวเราในอนาคต) ทำตามได้
(.venv) $ pip freeze > requirements.txt
```

```
# เครื่องใหม่ / เพื่อนร่วมทีม ใช้แค่บรรทัดเดียว
(.venv) $ pip install -r requirements.txt

# ออกจากกล่องเมื่อเลิกทำงาน
(.venv) $ deactivate
```

📖**อ่านว่า:** อ่าน `python3 -m venv .venv` ว่า: "ใช้ Python สร้างกล่องเปล่าชื่อ `.venv` ไว้ในโฟลเดอร์นี้" — กล่องนี้มี Python กับที่เก็บไลบรารีเป็นของตัวเอง แยกขาดจากเครื่อง. `activate` คือ "เดินเข้าไปอยู่ในกล่อง" ตั้งแต่นั้น `pip install` ทุกครั้งจะลงในกล่องนี้เท่านั้น ไม่ไปยุ่งกับโปรเจกต์อื่น

❌ อาการที่เกิดถ้าข้ามขั้นนี้ — และทำไมมันเจ็บ

- โปรเจกต์เก่าพังเพราะติดตั้งของใหม่ — โปรเจกต์ A ใช้ `pandas 1.5` ได้ดี วันหนึ่งคุณ `pip install` ไลบรารีใหม่ให้โปรเจกต์ B แล้วมันอัปเดต `pandas` เป็น 3.0 → โค้ดโปรเจกต์ A พังทันทีโดยที่คุณไม่ได้แตะมันเลย
- "เครื่องผมรันได้ แต่เครื่องคุณไม่ได้" — ไม่มีรายการว่าต้องใช้อะไรบ้าง เวอร์ชันไหน
- ลบของไม่ได้ — เมื่อทุกอย่างกองรวมกัน คุณจะไม่กล้าถอนอะไรเลยเพราะไม่รู้ว่ามีใครใช้อยู่

🔗กฎ: หนึ่งโปรเจกต์ = หนึ่ง `.venv` . ถ้าเห็น prompt ไม่มี `(.venv)` นำหน้า แปลว่าคุณกำลังติดตั้งลงเครื่องกลางอยู่ — ให้หยุดแล้ว `activate` ก่อน

🔵 [รอบสอง] เกร็ดที่ทำให้เจ็บน้อยลงอีก

- `pip freeze` จดทุกอย่างรวมของที่ติดมาเอง — ได้ไฟล์ยาวเหยียดที่อ่านไม่รู้เรื่อง . ทีมส่วนใหญ่จึงเขียน `requirements.txt` เองเฉพาะที่ใช้ตรง ๆ แล้วปล่อยให้ `pip` จัดการ dependency ที่เหลือ (หรือย้ายไปใช้ `uv` / `poetry` ที่แยก "ที่ฉันขอ" กับ "ที่ตามมา" ให้อัตโนมัติ)
- อย่า commit โฟลเดอร์ `.venv` ขึ้น git — มันใหญ่หลายร้อย MB และผูกกับ OS ของคุณ . ใส่ `.venv/` ใน `.gitignore` . สิ่งที่คุณ commit คือ `requirements.txt` เท่านั้น
- VS Code ต้องรู้ว่าใช้ตัวไหน — กด `Ctrl+Shift+P` → "Python: Select Interpreter" → เลือกตัวที่อยู่ใน `.venv` . ถ้าลืมขั้นนี้ คุณจะเจออาการงงที่สุด: รันใน terminal ได้ แต่กด F5 แล้ว `ModuleNotFoundError`
- ตรวจสอบว่าอยู่ในกล่องจริงไหม — `python -c "import sys; print(sys.prefix)"` ต้องขึ้น path ที่มี `.venv`

Running Example: ตลอด Part I — วิเคราะห์ราคา BTC

ทุกบทใน Part I จะต่อยอดจากตัวอย่างเดียวกัน: โหลดราคา BTC 1 ปี → คำนวณ return → หา signal → plot กราฟ

บท 6 เริ่มจาก variable พื้นฐาน → บท 9 จะเป็น DataFrame จริง → บท 10 จะ plot ออกมาให้เห็น

6.2 Variables และ Types

ใน Python ไม่ต้องประกาศ type — Python รู้เองว่าตัวแปรเก็บอะไร

```
# กำหนดค่าตัวแปร
price = 65000          # int (จำนวนเต็ม)
return_pct = 0.032     # float (ทศนิยม)
symbol = "BTCUSDT"    # str (ข้อความ)
is_long = True         # bool (True/False)

# ดู type
print(type(price))     # <class 'int'>
print(type(return_pct))# <class 'float'>
```

Type	ตัวอย่าง	ใช้กับ
int	65000 , -3 , 0	จำนวนหุ้น, วัน, index
float	0.032 , 65000.5	ราคา, ผลตอบแทน, volatility
str	"AAPL" , "2024-01-01"	ชื่อหุ้น, วันที่, label
bool	True , False	signal on/off, position open/close

ชื่อตัวแปรที่ดีใน Quant Code

- ใช้ snake_case: `daily_return` ไม่ใช่ `dailyReturn`
- ชื่อบอกความหมาย: `btc_price_usd` ดีกว่า `p`
- หลีกเลี่ยง `l` , `0` , `I` (สับสนกับ 1, 0)
- ค่า constant ใช้ตัวใหญ่: `RISK_FREE_RATE = 0.05`

6.3 อ่านโค้ดให้ออกเสียง — ทักษะแรกก่อนหัดเขียน

คนอ่านหนังสือออกเพราะรู้ว่าตัวอักษรแต่ละตัว "ออกเสียง" อย่างไร โค้ดก็เหมือนกัน — ทุกสัญลักษณ์มีเสียงอ่านในหัว ถ้ารู้เสียงอ่าน บรรทัดโค้ดจะกลายเป็นประโยคภาษาไทยธรรมดา และนี่คือทักษะที่สำคัญที่สุดในยุค AI: ต่อให้ไม่เคยเขียนเอง คุณก็อ่านโค้ดที่ AI เขียนให้ออกได้ เหมือนเด็กที่อ่านหนังสือออกก่อนจะเขียนเรียงความ

```
a = "hello world"
```

🔴**อ่านว่า:** เอา "hello world" มาใส่ใน a — เครื่องหมาย = ทำงานจากขวาไปซ้ายเสมอ: ค่าทางขวา ถูกใส่เข้า "กล่อง" ทางซ้าย

```
price = price + 10
```

🔴**อ่านว่า:** เอาค่า price เดิม บวก 10 แล้วใส่กลับเข้า price — บรรทัดนี้ไม่ใช่สมการคณิตศาสตร์ (ในคณิต price = price + 10 เป็นไปไม่ได้!) แต่คือคำสั่ง "อัปเดตค่า"

ตารางเสียงอ่าน — ใช้ตลอดทั้งเล่ม

สัญลักษณ์	อ่านว่า	ตัวอย่าง	อ่านทั้งบรรทัดว่า
=	"เอา...มาใส่ใน..."	<code>z = 2.5</code>	เอา 2.5 มาใส่ใน z
==	"ถามว่า...เท่ากับ...ไหม"	<code>z == 2.5</code>	ถามว่า z เท่ากับ 2.5 ไหม (ได้คำตอบ True/False)
!=	"ถามว่า...ไม่เท่ากับ...ไหม"	<code>signal != 0</code>	ถามว่า signal ไม่เท่ากับ 0 ใช่ไหม
>=	"ถามว่า...มากกว่าหรือเท่ากับ...ไหม"	<code>z >= 2.0</code>	ถามว่า z มากกว่าหรือเท่ากับ 2.0 ไหม
.	"ของ" (อ่านย้อนจากขวา)	<code>df.mean()</code>	ค่าเฉลี่ยของ df
[]	"หยิบตัวที่..."	<code>prices[0]</code>	หยิบตัวแรกจาก prices
() หลังชื่อ	"สั่งให้ทำงาน"	<code>calc_return()</code>	สั่งให้ calc_return ทำงาน
: ท้ายบรรทัด	"แล้วทำต่อไปนี่"	<code>if z > 2:</code>	ถ้า z มากกว่า 2 แล้วทำต่อไปนี่

หมายเหตุ: วงเล็บ () มีสองบทบาท — ถ้าอยู่หลังชื่อ (เช่น `print(...)`) แปลว่า "สั่งให้ทำงาน" แต่ถ้าครอบนิพจน์คณิตศาสตร์ (เช่น `(a - b) / b`) แปลว่า "ทำส่วนนี้ก่อน"

⚠️ **กับดักอันดับ 1 ของมือใหม่: = กับ ==**

```
if signal = 1: # ❌ SyntaxError! ใช้ "ใส่ค่า" ในที่ที่ต้อง "ถาม"
```

```
if signal == 1:    # ✓ "ถามว่า signal เท่ากับ 1 ไหม"
```

ง่าย ๆ: หนึ่งขีด = สิ่ง (ใส่ค่า), สองขีด = ถาม (เปรียบเทียบ) — โชคดีที่ Python ป้องกัน error ทันทีถ้าใช้ผิดใน if แต่ภาษาอื่นบางภาษาปล่อยผ่าน แล้ว bug จะตามหลอกหลอน

วิธีฝึกอ่านโค้ด

ทุกครั้งที่เจอโค้ดในเล่มนี้ ลองอ่านออกเสียงเป็นภาษาไทยที่ละบรรทัดก่อนดูคำอธิบาย ถ้าอ่านแล้วเป็นประโยคที่เข้าใจได้ แปลว่าคุณเข้าใจโค้ดบรรทัดนั้นแล้วจริง ๆ — จากบทนี้ไป จะมีกล่อง 📖 "อ่านว่า" แทรกไว้โค้ดสำคัญ เพื่อให้เทียบเสียงอ่านของตัวเองได้

6.4 Operators และ Expressions

```
# Arithmetic operators
price_today = 65000
price_yesterday = 63000

simple_return = (price_today - price_yesterday) / price_yesterday
print(f"Return: {simple_return:.2%}") # Return: 3.17%

# Operators ที่ใช้บ่อย
profit = 1000
tax_rate = 0.15
net = profit * (1 - tax_rate)    # = 850.0

days = 252
annual_vol = 0.03 * (days ** 0.5) # ** = ยกกำลัง (ทำไมไม่ใช่ ^ ดูกล่องด้านล่าง)

shares = 1000
remainder = shares % 100          # % = เศษจากหาร: 1000 % 100 = 0
lots = shares // 100              # // = หารปัดทิ้งทศนิยม: 1000 // 100 = 10
```

คำถามที่พบบ่อย: ทำไม `\*\*` ไม่ใช่ `^` แบบคณิต?

ใน Python เครื่องหมาย `^` ถูกสงวนไว้สำหรับ "XOR" (bitwise operation ที่ใช้ใน cryptography) จึงต้องใช้ `**` แทนการยกกำลัง — ไม่ผิดปกติ แค่ convention ของภาษา

สำหรับ `//` และ `%`: ถ้า `1000 / 100 = 10.0` (float) แล้ว `1000 // 100 = 10` (int, ปัดทิ้งทศนิยม) และ `1000 % 100 = 0` (เศษ) — สามสิ่งนี้เป็นชุดเดียวกัน: ผลหาร, ผลหารปัดลง, และเศษที่เหลือ

`range(1, 5)` ให้ค่า 1, 2, 3, 4 (ไม่รวมตัวสุดท้าย 5) — rule เดียวกับ slicing `[1:5]` นี่เป็น convention ของ Python เพื่อให้ `len(range(1, 5)) == 5 - 1 == 4` นับออกง่าย

6.5 f-string — แสดงผลแบบมีรูปแบบ

f-string คือวิธี "ประกอบข้อความ" สำหรับแสดงผล — ในเล่มนี้เราใช้ f-string เฉพาะใน `print()` ตอนแสดงผลสุดท้ายเท่านั้น การคำนวณจริงจะแยกเป็นตัวแปรชื่อชัด ๆ เสมอ เพื่อให้อ่านง่าย (AI ชอบยึดการคำนวณเข้าไปใน f-string เลย — เดี่ยวบทหลังจะฝึกอ่านแบบนั้น)

```
symbol = "BTC"
price = 65432.789
ret = 0.0317

# f-string: ใส่ตัวแปรใน { } ได้เลย
print(f"{symbol} ราคา: ${price:,.2f}")      # BTC ราคา: $65,432.79
print(f"Return: {ret:.2%}")                  # Return: 3.17%
print(f"Annualized: {ret * 252:.1f}%")        # Annualized: 8.0%
```

Format codes ที่ใช้บ่อยใน Quant

Code	ความหมาย	ตัวอย่าง	ผลลัพธ์
<code>:.2f</code>	ทศนิยม 2 ตำแหน่ง	<code>f"{65432.789:.2f}"</code>	<code>65432.79</code>
<code>:.0f</code>	ไม่มีทศนิยม	<code>f"{65432:.0f}"</code>	<code>65432</code>
<code>:,</code>	คั่นหลักพัน	<code>f"{65432:,}"</code>	<code>65,432</code>
<code>:.2%</code>	แปลงเป็น % ทศนิยม 2	<code>f"{0.0317:.2%}"</code>	<code>3.17%</code>
<code>:.4f</code>	ทศนิยม 4 ตำแหน่ง	<code>f"{0.0317:.4f}"</code>	<code>0.0317</code>

ตัวอย่าง: คำนวณและแสดงผล Trade Summary

```
entry_price = 63000.0
exit_price  = 65450.0
position_size = 0.5 # BTC

pnl = (exit_price - entry_price) * position_size
ret = (exit_price - entry_price) / entry_price

print(f"Entry:  ${entry_price:,.2f}")
print(f"Exit:   ${exit_price:,.2f}")
print(f"P&L:    ${pnl:,.2f}")
print(f"Return: {ret:.2%}")
```

► OUTPUT Entry: \$63,000.00 Exit: \$65,450.00 P&L: \$1,225.00 Return: 3.89%

► แบบฝึกหัด: คำนวณ Annualized Return

บทที่ 7: Control Flow & Functions

แก่นของบทนี้

Control flow คือการบอก program ว่า "ทำอะไรเมื่อเงื่อนไขแบบไหน" และ "ทำซ้ำกี่ครั้ง" — ใน Quant ใช้สร้าง signal logic, loop ผ่านข้อมูล, และ function ที่ reuse ได้ทั้ง codebase

7.1 เงื่อนไขคืออะไร — คำถามที่ตอบได้แค่ True/False

ก่อนรู้จัก if ต้องเข้าใจก่อนว่า "เงื่อนไข" คืออะไร: เงื่อนไขคือคำถามที่คำตอบมีแค่ 2 แบบ — True (จริง) หรือ False (เท็จ)

```
z = 2.3

z > 2.0      # คำถาม: z มากกว่า 2.0 ไหม? → คำตอบ: True
z == 2.0     # คำถาม: z เท่ากับ 2.0 ไหม? → คำตอบ: False
z < -2.0     # คำถาม: z น้อยกว่า -2.0 ไหม? → คำตอบ: False
```

คำตอบ True/False เป็น "ค่า" จริง ๆ ใน Python — เอามาใส่ตัวแปรได้เหมือนตัวเลข:

```
is_overbought = z > 2.0      # เอาคำตอบ (True) มาใส่ใน is_overbought
print(is_overbought)        # True
```

📌อ่านว่า: ถามว่า z มากกว่า 2.0 ไหม แล้วเอาคำตอบมาใส่ใน is_overbought — สังเกตว่า = (ใส่ค่า) กับ > (ถาม) ทำงานคนละหน้าที่ในบรรทัดเดียวกัน: ฝั่งขวาก่อนได้คำตอบแล้วค่อยใส่

Comparison Operators — เครื่องมือสร้างคำถาม

Operator	อ่านว่า	ตัวอย่าง
==	เปรียบเทียบกับ...เท่ากับ... (ได้ True/False)	signal == "LONG"
!=	ไม่เท่ากับ...ใช่ไหม	position != 0
> , <	มากกว่า/น้อยกว่า...ไหม	z > 2.0
>= , <=	อย่างน้อย / ไม่เกิน...ไหม	drawdown <= -0.1

7.2 if / elif / else — เครื่องจักรถามคำถามตามลำดับ

if/elif/else คือเครื่องจักรที่ถามคำถามจากบนลงล่างทีละข้อ — เจอข้อไหนตอบ "ใช่" ทำงานของข้อนั้นแล้วออกจากเครื่องทันที ข้อที่เหลือไม่ถูกถามอีกเลย

```
z = 2.3

if z > 2.0:          # ถามข้อ 1: z มากกว่า 2.0 ไหม?
    signal = "SHORT" #   ใช่ → ทำบรรทัดนี้ แล้วออกเลย ไม่ถามต่อ
elif z < -2.0:       # ถามข้อ 2 (เฉพาะเมื่อข้อ 1 ตอบ "ไม่")
    signal = "LONG"  #   ใช่ → ทำบรรทัดนี้ แล้วออก
elif abs(z) < 0.5:   # ถามข้อ 3 (เฉพาะเมื่อข้อ 1 และ 2 ตอบ "ไม่")
    signal = "EXIT"  #   ใช่ → ทำบรรทัดนี้ แล้วออก
else:                # ทุกข้อข้างบนตอบ "ไม่" ทั้งหมด
    signal = "HOLD"  #   ทำบรรทัดนี้ (else ไม่มีคำถาม = "ที่เหลือทั้งหมด")

print(signal)
```

► OUTPUT **SHORT**

🔴**อ่านว่า:** ถ้า z มากกว่า 2.0 แล้วเอา "SHORT" ใส่ใน signal — ไม่งั้นถ้า z น้อยกว่า -2.0 แล้วเอา "LONG" ใส่ใน signal — ไม่งั้นถ้าขนาดของ z น้อยกว่า 0.5 แล้วเอา "EXIT" ใส่ — ไม่งั้น (ที่เหลือทั้งหมด) เอา "HOLD" ใส่

เส้นทางของ $z = 2.3$ ในเครื่องจักรนี้:

ข้อ 1: $z > 2.0$? ($2.3 > 2.0$)

| ตอบ "ใช่" ↓

signal = "SHORT" → ออกจากเครื่องทันที

ข้อ 2, ข้อ 3, else ไม่ถูกถามเลย แม้ว่า $\text{abs}(2.3) < 0.5$ จะเป็น False อยู่แล้วก็ตาม

⚠ elif ไม่เท่ากับ if สองอัน — ตัวอย่างที่ผลต่างกันจริง

```
z = 3.0
# — แบบ elif: ถามต่อเมื่อข้อบนตอบ "ไม่" —
if z > 2.0:
    action = "SHORT" # เข้าข้อนี้ แล้วออกเลย
elif z > 1.0:
    action = "WARN"  # ไม่ถูกถาม
# ผล: action = "SHORT" ✓

# — แบบ if สองอัน: ถามทุกข้อเสมอ —
if z > 2.0:
```

```

    action = "SHORT"      # เข้าซื้อ
if z > 1.0:
    action = "WARN"      # ถูกถามอีก! 3.0 > 1.0 จริง → ทับค่าเดิม
# ผล: action = "WARN" ❌ SHORT หายไปเฉย ๆ

```

ถ้าเงื่อนไขเป็น "ทางเลือกที่เกิดได้ทีละอย่าง" (signal มีได้ค่าเดียว) ต้องใช้ `elif` เสมอ — bug ชนิดนี้ Python ไม่ฟ้อง error โปรแกรมรันได้ปกติแต่ให้ signal ผิด ซึ่งใน trading คือเสียเงินจริง

ลำดับเงื่อนไขสำคัญ — เช็คข้อแคบก่อนข้อกว้าง

```

# ❌ ผิด: ข้อกว้าง (z > 1) อยู่บน → "กิน" ทุกกรณี z > 2 ไปด้วย
if z > 1.0:
    level = "WARN"      # z = 3.0 ก็เข้าซื้อ! ไม่มีวันถึง EXTREME
elif z > 2.0:
    level = "EXTREME"   # โค้ดตายซาก — ไม่มีทางถูกเรียก

# ✓ ถูก: ข้อแคบ/รุนแรงกว่า ขึ้นก่อน
if z > 2.0:
    level = "EXTREME"
elif z > 1.0:
    level = "WARN"

```

เชื่อมหลายเงื่อนไข: and / or / not

ตัวเชื่อม	อ่านว่า	ผลเป็น True เมื่อ
and	"และ"	จริงทั้งคู่
or	"หรือ"	จริงอย่างน้อยหนึ่งข้อ
not	"ไม่/กลับด้าน"	ของเดิมเป็น False

```

# เข้า trade เมื่อ: z สูงพอ "และ" ยังไม่มี position
if z > 2.0 and position == 0:
    position = -1

# ออกจาก trade เมื่อ: z กลับใกล้ 0 "หรือ" ขาดทุนเกิน limit
if abs(z) < 0.5 or loss > max_loss:
    position = 0

```

📌 **อ่านว่า:** ถ้า z มากกว่า 2.0 และ position เท่ากับ 0 (ทั้งสองข้อต้องจริง) แล้วเอา -1 ใส่ใน position

⚠ กับดักการเขียนเงื่อนไข 2 จุด

```
# 1) เงื่อนไขช่วง — ต้องเขียนตัวแปรซ้ำทั้งสองฝั่ง
if z > 1.0 and < 3.0:      # ❌ SyntaxError
if z > 1.0 and z < 3.0:    # ✓
if 1.0 < z < 3.0:          # ✓ Python ยอมให้เขียนแบบคณิตได้ด้วย

# 2) indent (ย่อหน้า) กำหนดว่าบรรทัดไหนอยู่ "ใน" if
if z > 2.0:
    signal = -1
    log_trade()             # อยู่ใน if — ทำเฉพาะเมื่อ z > 2.0

if z > 2.0:
    signal = -1
log_trade()                 #อยู่นอก if — ทำทุกครั้ง! ต่างแค่ย่อหน้าเดียว
```

7.3 for loop — วนซ้ำ

for loop คือคำสั่ง "หยิบของจากรายการมาทีละตัว แล้วทำชุดคำสั่งเดิมซ้ำกับทุกตัว"

```
prices = [63000, 65000, 64500]

for p in prices:
    print(p)                # หยิบสมาชิกจาก prices มาใส่ p ทีละตัว
                           # ทำบรรทัดนี้กับ p ทุกรอบ
```

📌 **อ่านว่า:** สำหรับแต่ละตัวใน prices — เอาตัวนั้นมาใส่ใน p แล้วทำต่อไปนี่: print ค่า p

ดูการทำงานทีละรอบ — เทคนิคสำคัญ: เวลาจบ loop ให้ "เดินเครื่องด้วยมือ" ตามตาราง — จดทั้ง *ค่าตัวแปร loop (i)* และ *state ของ list ที่กำลังสะสม (returns)*:

รอบที่	i	prices[i]	returns (list สะสม)
1	1	65000	[0.0317]
2	2	64500	[0.0317, -0.0077]
3	3	66000	[0.0317, -0.0077, 0.0233]
4	4	65800	[0.0317, -0.0077, 0.0233, -0.0030] → loop จบ

ตัวอย่างจริง: คำนวณ return รายวัน — ต้องใช้ "ราควันนี้" คู่กับ "ราคาเมื่อวาน" จึง loop ด้วยตำแหน่ง (index) แทน:

```

prices = [63000, 65000, 64500, 66000, 65800]

returns = [] # เตรียม list เปล่าไว้เก็บผล
for i in range(1, len(prices)): # i ไหลจาก 1, 2, 3, 4 (เริ่มวันที่สอง)
    today = prices[i] # หยิบราคาวันนี้
    yesterday = prices[i-1] # หยิบราคาเมื่อวาน (ตัวก่อนหน้า)
    r = (today - yesterday) / yesterday # สูตร simple return
    returns.append(r) # เก็บผลต่อท้าย list

print(returns) # [0.0317, -0.0077, 0.0233, -0.0030]

```

```

# enumerate: ได้ทั้งตำแหน่งและค่า พร้อมกัน
for i, price in enumerate(prices):
    print(f"วันที่ {i+1}: ${price:,}")

# zip: loop สองรายการพร้อมกัน (จับคู่ตัวต่อตัว)
symbols = ["BTC", "ETH", "SOL"]
weights = [0.5, 0.3, 0.2]
for sym, w in zip(symbols, weights):
    print(f"{sym}: {w:.0%}")

```

while loop — วนจนกว่าเงื่อนไขจะเป็นเท็จ

for loop กับ while loop แตกต่างกันที่ "รู้ล่วงหน้าไหมว่าจะวนกี่รอบ":

	for loop	while loop
ใช้เมื่อ	รู้จำนวนรอบล่วงหน้า	ไม่รู้จำนวนรอบ — วนจนเงื่อนไขเปลี่ยน
หยุดเมื่อ	หมด list / range	เงื่อนไขกลายเป็น False
ตัวอย่าง Quant	คำนวณ return 252 วัน (รู้ว่า 252 วัน)	เทรดต่อจนทุนหมด (ไม่รู้วันที่วัน)

```

equity = 100000 # เงินเริ่มต้น 100,000 บาท
max_loss = 90000 # ถ้าเงินเหลือน้อยกว่า 90,000 หยุดเทรด
daily_return = -0.004 # สมมติขาดทุนวันละ 0.4% (จงใจให้ติดลบ เพื่อให้ loop มีวันจบ — จริง ๆ จะได้จาก
day = 0 # ตัวนับรอบ

while equity > max_loss: # ก่อนทุกรอบ ถาม: equity ยังมากกว่า max_loss ไหม?
    day = day + 1 # นับรอบที่เทรด
    profit = equity * daily_return # คำนวณกำไร/ขาดทุนวันนี้ (ติดลบ = ขาดทุน)
    equity = equity + profit # อัปเดตเงินทุน (ลดลงเรื่อย ๆ)
    print(f"รอบ {day}: equity = {equity:.2f}") # ไม่ → ออกจาก loop (หยุดเทรด)

print("หยุดเทรด — ขาดทุนถึง limit")

```



```
# comprehension + if = กรองของ
loss_days = [r for r in returns if r < 0]
print(f"วันขาดทุน: {len(loss_days)} วัน")
```

🔗**อ่านว่า:** สร้าง list จาก r แต่ละตัวใน returns เฉพาะตัวที่ r น้อยกว่า 0 — แล้วเอามาใส่ใน loss_days

🧠 เมื่อ AI ใช้ if ใน comprehension + ternary — 2 รูปแบบที่ต่างกัน

```
# รูปแบบที่ 1: กรอง (filter) — if อยู่หลัง for
loss_days = [r for r in returns if r < 0]
# รูปแบบที่ 2: เลือกค่า (ternary) — if อยู่หน้า for
labels = ["loss" if r < 0 else "gain" for r in returns]
```

ทั้งสองรูปแบบมี if แต่ตำแหน่งต่างกัน → ความหมายต่างกันสิ้นเชิง:

1. **if หลัง for** = กรองของ: "เอาเฉพาะ r ที่ $r < 0$ " → list สั้นลง
2. **if...else หน้า for** = เลือกค่า: "ถ้า $r < 0$ ให้เป็น 'loss' ไม่งั้น 'gain'" → list ยาวเท่าเดิม แค่เปลี่ยนค่า

จำง่าย: ก่อน for = เลือกว่าแต่ละตัวจะเป็นอะไร, หลัง for = กรองว่าตัวไหนเข้าหรือออก

7.5 Functions — def

```
# นิยาม function
def calc_return(price_now, price_prev):
    """คำนวณ simple return ระหว่างสองราคา"""
    return (price_now - price_prev) / price_prev

# เรียกใช้
r = calc_return(65000, 63000)
print(f"Return: {r:.2%}") # Return: 3.17%

# Default arguments
def annualize(daily_vol, periods=252):
    """แปลง daily volatility เป็น annualized"""
    return daily_vol * (periods ** 0.5)

print(annualize(0.03)) # 0.4762 (252 วัน = หุ้น)
print(annualize(0.03, 365)) # 0.5733 (365 วัน = crypto)
```

Docstring — เขียนทุก function ใน Quant code

บรรทัดแรกใน function (ในเครื่องหมาย `"""`) คือ docstring อธิบาย function นั้น
ช่วยเมื่อ copy code ไปให้ AI ตรวจ — AI จะเข้าใจ intent ได้ถูกต้อง และช่วย debug ได้แม่นยำขึ้น

ตัวอย่าง: Z-score Function แบบ Reusable

```
def zscore(series, window=60):  
    """  
    คำนวณ rolling z-score ของ series  
    Args:  
        series: list หรือ numpy array ของราคา/spread  
        window: rolling window (default 60 วัน)  
    Returns:  
        list ของ z-score  
    """  
    results = [] # เตรียม list เก็บผลลัพธ์  
    for i in range(len(series)): # เดินผ่านข้อมูลทีละวัน  
        if i < window: # วันแรก ๆ ข้อมูลย้อนหลังยังไม่ครบ window  
            results.append(None) # → ใส่ None แทน (คำนวณไม่ได้)  
            continue # ข้ามไปวันถัดไปเลย  
        window_data = series[i-window:i] # หยิบข้อมูล window วันล่าสุด  
        mean = sum(window_data) / window # ค่าเฉลี่ยของหน้าต่างนั้น  
        variance = sum((x - mean)**2 for x in window_data) / window # ความแปรปรวน  
        std = variance ** 0.5 # ส่วนเบี่ยงเบนมาตรฐาน = รากของ variance  
        if std == 0: # กันหารด้วยศูนย์ (ราคาริ่งสนิท)  
            results.append(0)  
        else:  
            z = (series[i] - mean) / std # สูตร z-score: ห่างจากค่าเฉลี่ยกี่ std  
            results.append(z)  
    return results  
  
# ทดสอบ  
spread = [100, 102, 98, 105, 99, 103, 110, 97, 101, 104]  
z = zscore(spread, window=5)  
print(z[-3:]) # z-score 3 วันล่าสุด
```

AI เขียนฟังก์ชันเดียวกันนี้ใน 1 บรรทัด — ฝึกอ่านกัน

```
z = (s - s.rolling(60).mean()) / s.rolling(60).std()
```

กล่องนี้คือ "ตัวอย่างล่องหน้า" — `.rolling()` จะเรียนละเอียดในบทที่ 9 ตอนนี้แค่ฝึกอ่านโครงสร้างให้ออก

(`s` คือ pandas Series — จะเรียนบทที่ 9) อ่านทีละชิ้นจากในออกนอก:

1. `s.rolling(60)` — "หน้าต่างเลื่อน 60 วัน ของ s" (จุด = "ของ")
2. `s.rolling(60).mean()` — "ค่าเฉลี่ย ของ หน้าต่าง 60 วัน" = ตัวแปร `mean` ในฟังก์ชันเรา
3. `s.rolling(60).std()` — "std ของหน้าต่าง 60 วัน" = ตัวแปร `std` ของเรา
4. `(s - mean) / std` — สูตร z-score เดิมเป๊ะ แค่ทำกับทุกวันพร้อมกัน (ไม่ต้อง loop เอง)

โค้ด 15 บรรทัดของเรากับบรรทัดเดียวนี่ *ทำสิ่งเดียวกัน* — pandas ซ่อน loop ไว้ข้างใน (และเร็วกว่า ~100 เท่า) เราเขียนแบบยาวก่อนเพื่อให้รู้ว่า "ข้างในมันทำอะไร" พอไปบทที่ 9 จะใช้แบบสั้นได้อย่างมั่นใจ

7.6 lambda — Function ย่อ

```
# lambda: function 1 บรรทัด ไม่มีชื่อ
calc_ret = lambda p_now, p_prev: (p_now - p_prev) / p_prev

# ใช้บ่อยกับ sorted() และ map()
trades = [("BTC", 500), ("ETH", -200), ("SOL", 300)]
by_pnl = sorted(trades, key=lambda t: t[1], reverse=True)
print(by_pnl) # [('BTC', 500), ('SOL', 300), ('ETH', -200)]
```

► แบบฝึกหัด: เขียน function คำนวณ Sharpe Ratio

7.7 yield — ฟังก์ชันที่ส่งของที่ละชิ้น

`return` ที่เพิ่งเรียนไปมีลักษณะเด็ดขาด: **ส่งของออกไปแล้วฟังก์ชันจบทันที** . มีคำสั่งอีกตัวที่ทำเกือบเหมือนกันแต่ต่างกันตรงจุดนี้จุดเดียว

ต่างกันแค่ประโยคเดียว

```
return = "นี่ของ แล้วฉันจบแล้ว"
yield = "นี่ของชิ้นหนึ่ง ฉันหยุดตรงนี้ ถ้าอยากได้อีกค่อยเรียก"
```

ฟังก์ชันที่มี `yield` อยู่ข้างในเรียกว่า **generator** — มันจำได้ว่าค้างอยู่บรรทัดไหน แล้ววิ่งต่อจากตรงนั้นเมื่อถูกขอขึ้นถัดไป

☛ **อ่านว่า:** เปรียบกับการสั่งอาหาร: `return` คือยกมาทั้งหมดวางบนโต๊ะ — ได้ครบแน่นอน แต่ต้องมีที่วางทั้งหมด .

`yield` คือตักให้ทีละจาน — โต๊ะเล็กก็กินได้ และถ้าอิมก่อนก็ไม่ต้องตักที่เหลือ . ในงานข้อมูลราคาที่มีเป็นล้านแถว ความต่างนี้คือความต่างระหว่าง "รันได้" กับ "แรมเต็ม"

```
import sys
```

```
def prices_list(n):
    """แบบ return — สร้างลิสต์ทั้งก้อนแล้วค่อยส่งกลับ"""
    out = []
    for i in range(n):
        out.append(65000 + i * 0.5)
    return out

def prices_gen(n):
    """แบบ yield — ส่งทีละตัว ไม่เก็บอะไรไว้เลย"""
    for i in range(n):
        yield 65000 + i * 0.5

N = 1_000_000
lst = prices_list(N)
gen = prices_gen(N)

print(f"list      : กินหน่วยความจำ {sys.getsizeof(lst) / 1024**2:>7.1f} MB")
print(f"generator : กินหน่วยความจำ {sys.getsizeof(gen) / 1024**2:>7.4f} MB")

# ใช้งานเหมือนกันทุกอย่าง — วนด้วย for ก็ได้ ส่งเข้า sum() ก็ได้
print(f"sum 5 ตัวแรก · list={sum(prices_list(5)):.1f} "
      f"· gen={sum(prices_gen(5)):.1f}")
```

► OUTPUT list : กินหน่วยความจำ 8.1 MB generator : กินหน่วยความจำ 0.0002 MB sum 5 ตัวแรก · list=325,005.0 · gen=325,005.0

📌อ่านว่า: ตัวเลข **8.1 MB** เทียบกับ **0.0002 MB** ไม่ได้แปลว่า generator "บีบอัดข้อมูล" — มันไม่ได้เก็บข้อมูลไว้เลยสักตัว · สิ่งที่มีเก็บคือ "สูตรว่าจะสร้างตัวถัดไปยังไง" กับ "ตอนนี้ค้างอยู่ตรงไหน" เท่านั้น · ราคาที่จ่ายคือถ้าอยากได้ตัวที่ 500,000 ต้องเดินผ่าน 499,999 ตัวแรกก่อน — เข้าถึงแบบสุ่ม (gen[500000]) ทำไม่ได้

⚠ generator ใช้ได้ครั้งเดียว — กับดักที่ไม่มี error เตือน

```
g = prices_gen(5)
print(f"รอบแรก sum(g) = {sum(g):.1f}")
print(f"รอบสอง sum(g) = {sum(g):.1f}") # ← generator หมดแล้ว
```

► OUTPUT รอบแรก sum(g) = 325,005.0 รอบสอง sum(g) = 0.0

รอบสองได้ **0.0** ไม่ใช่ error · เพราะ generator เดินไปจนสุดแล้ว การขอต่อจึงได้ "ไม่มีอะไรเลย" ซึ่ง sum() ของความว่างเปล่าคือ 0

ในงานจริงกับดักนี้หน้าตาแบบนี้:

— ตัวอย่างประกอบ —

```
returns = (r for r in load_prices() if r > 0) # generator expression
n       = sum(1 for _ in returns)           # นับ → กินหมดแล้ว
mean    = sum(returns) / n                 # ❌ ได้ 0 · แล้วหารต่อได้ 0.0
```

ไม่มี exception · ไม่มี warning · ได้ค่าเฉลี่ยเป็น 0.0 แล้วไหลเข้าไปในรายงานอย่างสงบ

🚫 **วิธีกัน:** ถ้าต้องใช้ข้อมูลชุดเดิมมากกว่าหนึ่งรอบ ให้แปลงเป็น list ก่อนด้วย `list(gen)` แล้วยอมเสียหน่วยความจำไป · generator เหมาะกับงานที่ไหลผ่านครั้งเดียวจบ เท่านั้น

● [รอบสอง] ที่ไหนในสาย quant ที่ generator เป็นคำตอบจริง ๆ

- **ไฟล์ที่ใหญ่กว่าแรม** — tick data ปีเดียวเป็นสิบลบ GB · `for line in open(f)` คือ generator อยู่แล้วโดยธรรมชาติ อ่านทีละบรรทัดไม่เคยโหลดทั้งไฟล์ · เขียน `open(f).readlines()` เมื่อไรคือโหลดทั้งไฟล์เข้าแรมทันที
- **ข้อมูลที่ยังไม่จบ** — ราคาสดจาก WebSocket ไม่มี "ตัวสุดท้าย" ให้ใส่ list · generator เป็นรูปแบบที่ตรงกับธรรมชาติของมัน (Part VI จะเจอญาติของมันคือ `async for`)
- **ต่อท่อเป็นทอด ๆ** — อ่านไฟล์ → กรอง → แปลง → รวมยอด ถ้าแต่ละขั้นคืน list จะสร้างสำเนาข้อมูลใหม่ทุกขั้น · ถ้าเป็น generator ทั้งสาย ข้อมูล 1 แถวจะไหลผ่านทุกขั้นแล้วถูกทิ้งไป ก่อนแถวถัดไปจะเริ่ม

และที่ไหนที่ไม่ควรใช้: ถ้าข้อมูลใส่แรมได้สบายและต้องวนหลายรอบ — `list` หรือ `pd.Series` ชะงืดทั้งความง่ายและความเร็ว · **อย่าใช้ generator เพราะมันดูโปร** ใช้เพราะข้อมูลใหญ่เกินหรือไหลไม่จบเท่านั้น

เกร็ดที่โยกกลับไปหัวข้อที่แล้ว: เขียน `[x for x in ...]` ได้ list · เปลี่ยนวงเล็บเหลี่ยมเป็นวงเล็บโค้ง (`x for x in ...`) ได้ **generator expression** — ต่างกันแค่วงเล็บ แต่ตัวหนึ่งกินแรมทั้งก้อน อีกตัวไม่กินเลย

บทที่ 8: Data Structures

แก่นของบทนี้

Python มี 4 data structure หลัก — **list** (รายการเรียงลำดับ), **dict** (key-value), **tuple** (ค่าคงที่), **set** (ค่าไม่ซ้ำ) แต่ละอย่างมีจุดแข็งต่างกัน รู้จักเลือกใช้ถูกตัว code จะสั้นและเร็วขึ้นมาก

8.1 list — รายการที่เรียงลำดับ

```
prices = [63000, 65000, 64500, 66000, 65800]

# Indexing (เริ่มจาก 0)
print(prices[0])    # 63000   (ตัวแรก)
print(prices[-1])   # 65800   (ตัวสุดท้าย - เลขติดลบ = นับจากท้าย)

# Slicing [start:stop:step]
print(prices[1:4])   # [65000, 64500, 66000]
print(prices[:3])    # [63000, 65000, 64500] (3 ตัวแรก)
print(prices[::2])   # [63000, 64500, 65800] (ทุก 2 ตัว)

# เพิ่ม/ลบ
prices.append(67000)    # เพิ่มต่อท้าย
prices.insert(0, 62000) # แทรกที่ตำแหน่ง 0
last = prices.pop()     # ดึงตัวสุดท้ายออก

# ข้อมูลพื้นฐาน
print(len(prices))      # จำนวนสมาชิก
print(min(prices), max(prices)) # ต่ำสุด สูงสุด
print(sum(prices) / len(prices)) # ค่าเฉลี่ย
```

8.2 dict — Key-Value Store

dict คือโครงสร้างที่ใช้มากที่สุดในการเก็บข้อมูลตลาด — key คือชื่อ/symbol, value คือข้อมูล

```
# OHLCV ของวันหนึ่ง
candle = {
    "open": 64000.0,
    "high": 66500.0,
    "low": 63800.0,
    "close": 65450.0,
```

```

    "volume": 28500.0
}

# เข้าถึงค่า
print(candle["close"])          # 65450.0
print(candle.get("vwap", None)) # None (ไม่มี key นั้น - ไม่ error)

# เพิ่ม/อัปเดต
candle["body"] = candle["close"] - candle["open"] # = 1450.0

# loop ผ่าน dict
for key, value in candle.items():
    print(f"{key:10s}: {value:,.1f}")

# Portfolio เป็น dict ของ dict
portfolio = {
    "BTC": {"size": 0.5, "entry": 63000, "side": "long"},
    "ETH": {"size": 2.0, "entry": 3200, "side": "long"},
    "SOL": {"size": 10.0, "entry": 145, "side": "short"},
}

```

8.3 tuple — ค่าคงที่

```

# tuple เหมือน list แต่ immutable (เปลี่ยนไม่ได้)
config = (2.0, 0.5, 60) # (entry_z, exit_z, window)
entry_z, exit_z, window = config # unpacking

# ใช้เป็น key ของ dict ได้ (list ทำไม่ได้)
pair_params = {
    ("BTC", "ETH"): {"beta": 0.85, "half_life": 4.2},
    ("BTC", "SOL"): {"beta": 1.20, "half_life": 6.8},
}

print(pair_params[("BTC", "ETH")]["beta"]) # 0.85

```

8.4 set — ค่าไม่ซ้ำ

```

# set เก็บเฉพาะค่าที่ไม่ซ้ำ
signals_day1 = {"AAPL", "MSFT", "NVDA"}
signals_day2 = {"MSFT", "NVDA", "TSLA"}

both_days = signals_day1 & signals_day2 # intersection = {"MSFT", "NVDA"}
either_day = signals_day1 | signals_day2 # union
only_day1 = signals_day1 - signals_day2 # difference = {"AAPL"}

```

```
# ลบ duplicate ออกจาก list
trades = ["BTC", "ETH", "BTC", "SOL", "ETH", "BTC"]
unique_symbols = list(set(trades)) # ["BTC", "ETH", "SOL"]
```

เลือก Data Structure ไหน?

ถ้าต้องการ	ใช้	เหตุผล
เก็บ series ราคาเรียงเวลา	list	เรียงลำดับ, append ง่าย
เก็บ OHLCV ของ 1 candle	dict	access ด้วยชื่อ field ได้
ค่า config ที่ไม่เปลี่ยน	tuple	immutable, ใช้เป็น dict key ได้
รายการ symbol ที่เคยเทรด	set	ลบ duplicate อัตโนมัติ
ข้อมูลตลาดหลายวัน	pd.DataFrame	Part 1 บทที่ 9

ตัวอย่าง: สร้าง Return Map ของหลายเหรียญ

```
close_prices = {
    "BTC": [63000, 65000, 64500],
    "ETH": [3100, 3200, 3150],
    "SOL": [140, 148, 145],
}

# แบบที่เราเขียน: loop ผ่านทุกเหรียญ คำนวณ return ล่าสุด
latest_returns = {} # เตรียม dict เปล่า
for sym, prices in close_prices.items(): # หยิบ (ชื่อเหรียญ, list ราคา) ทีละคู่
    last = prices[-1] # ราคาล่าสุด (ตัวสุดท้าย)
    prev = prices[-2] # ราคาก่อนหน้า (ตัวรองสุดท้าย)
    latest_returns[sym] = (last - prev) / prev # เก็บ return โดยใช้ชื่อเหรียญเป็น key

for sym, r in latest_returns.items():
    print(f"{sym}: {r:.2%}")
```

► OUTPUT BTC: -0.77% ETH: -1.56% SOL: -2.03%

```
latest_returns = {
    sym: (prices[-1] - prices[-2]) / prices[-2]
    for sym, prices in close_prices.items()
}
```

สูตรเดียวกับ list comprehension: อ่านบรรทัด **for** ก่อน แล้วย้อนขึ้นไปดูว่าแต่ละรอบสร้างอะไร

1. `for sym, prices in close_prices.items()` — loop เดิม: หยิบ (ชื่อเหรียญ, ราคา) ทีละคู่
2. `sym: (prices[-1] - prices[-2]) / prices[-2]` — แต่ละรอบสร้างคู่ `key: value` = ชื่อเหรียญ : return ล่าสุด
3. `{ ... }` ปีกกาครอบ — เก็บทุกคู่ลง dict (ปีกกา = dict, วงเล็บเหลี่ยม = list)

ความหมายเหมือนโค้ด 5 บรรทัดข้างบนทุกประการ

► แบบฝึกหัด: สร้าง Portfolio Summary

8.5 `with` — เปิดแล้วปิดให้เองไม่ต้องจำ

บทหน้าจะเริ่มอ่านไฟล์จริง · ก่อนไปถึงตรงนั้นมีไวยากรณ์หนึ่งตัวที่ต้องรู้ก่อน เพราะจะเจอไปตลอดทั้งเล่ม — และจะกลับมาในรูปแบบ `async with` ที่ Part VI

```
# ❌ แบบที่ลืมนง่าย — ต้องปิดเอง
f = open("trades.csv", "w")
f.write("symbol,pnl\n")
f.write("BTC,1250\n")
f.close() # ถ้าลืมบรรทัดนี้ ข้อมูลอาจไม่ถูกเขียนลงดิสก์จริง

# ❌ แย่กว่านั้น — ถ้าเกิด error ก่อนถึง close() ไฟล์จะค้างเปิดไว้
# (คอมเมนต์ไว้เพราะรันแล้วพังจริง — ลองเอา # ออกดูได้)
# f = open("trades.csv", "w")
# f.write(str(1 / 0)) # ZeroDivisionError -> ไม่มีใครไปถึง f.close()
# f.close()

# ✅ แบบที่ควรใช้ — with ปิดให้เสมอ ไม่ว่าจะจบแบบไหน
with open("trades.csv", "w") as f:
    f.write("symbol,pnl\n")
    f.write("BTC,1250\n")
# พ้นย่อหน้านั้นเมื่อไหร่ = ไฟล์ถูกปิดแล้วเรียบร้อย แม้ข้างในจะ error ก็ตาม

with open("trades.csv") as f:
    print(f.read())
```

► OUTPUT `symbol,pnl BTC,1250`

📖 **อ่านว่า:** อ่าน `with open("trades.csv", "w") as f:` ว่า: "ยืมไฟล์นี้มาใช้ เรียกมันว่า `f` — พอออกจากย่อหน้านี้ให้คืนของอัตโนมัติ". จุดสำคัญคือคำว่า *อัตโนมัติ*: จบปกติก็ปิด, เกิด error กลางทางก็ปิด, `return` ออกไปกลางคันก็ปิด — ไม่มีเส้นทางไหนที่ไฟล์จะค้างเปิด

ทำไมเรื่อง "ลืมปิด" ถึงสำคัญกว่าที่คิด

- **ไฟล์:** ข้อมูลที่ `write()` ไปอาจยังค้างใน buffer ไม่ลงดิสก์จริงจนกว่าจะปิด — โปรแกรมจบแล้วไฟล์ว่างเปล่า
- **จำนวนไฟล์ที่เปิดพร้อมกันมีเพดาน:** ลูปที่เปิดไฟล์ทีละอันโดยไม่ปิด พอวนหลายพันรอบจะได้ `OSError: Too many open files`
- **ของอย่างอื่นก็ใช้ `with` ได้:** การเชื่อมต่อฐานข้อมูล, network socket, lock — อะไรก็ตามที่ "เปิดแล้วต้องปิด". ใน Part VI เราจะใช้ `async with` กับการเชื่อมต่อ WebSocket ด้วยเหตุผลเดียวกันปะ

🔵 [รอบสอง] ในงาน quant จริงคุณจะเห็น `with` ที่ไหนบ้าง

ส่วนใหญ่ pandas จัดการไฟล์ให้แล้ว (`pd.read_csv("x.csv")` เปิดและปิดให้เอง) — ที่ยังต้องเขียน `with` เองคือ ① การเชื่อมต่อฐานข้อมูล ② การตั้งค่าชั่วคราว ③ การเขียน Excel หลาย sheet (`pd.ExcelWriter`) . สองข้อแรกลองได้เลย:

```
import sqlite3
import pandas as pd

# ① เปิด/ปิด connection ฐานข้อมูล — ":memory:" คือ DB ชั่วคราวในแรม
with sqlite3.connect(":memory:") as conn:
    pd.DataFrame({"symbol": ["BTC", "ETH"],
                  "close": [65000, 3200]}).to_sql("ohlcv", conn, index=False)
    df = pd.read_sql("SELECT * FROM ohlcv WHERE close > 10000", conn)
print(df)

# ② ตั้งค่าชั่วคราวแล้วคืนค่าเดิมอัตโนมัติ
big_df = pd.DataFrame({"x": range(100)})
print("นอก with :", pd.get_option("display.max_rows"))
with pd.option_context("display.max_rows", 5):
    print("ใน with :", pd.get_option("display.max_rows"))
print("พ้น with :", pd.get_option("display.max_rows"), "<- คืนค่าเดิมให้เอง")
```

► OUTPUT `symbol close 0 BTC 65000` นอก with : 60 ใน with : 5 พ้น with : 60 <- คืนค่าเดิมให้เอง

💡 สร้างของแบบนี้เองก็ได้ด้วย `contextlib.contextmanager` — ใช้บ่อยตอนทำตัวจับเวลา หรือตอนต้องเปิด/ปิดการเชื่อมต่อ exchange ในโค้ด live . หลักคิดเดียวกันทั้งหมด: จับคู่ "เปิด" กับ "ปิด" ให้อยู่ในโครงสร้าง

เดียว แทนที่จะฟังความจำของคนเขียน

บทที่ 9: NumPy & Pandas

แก่นของบทนี้

NumPy และ Pandas เป็น "engine" ของ Quant Python — NumPy ทำ math บน array เร็วกว่า loop ธรรมดา 100x+, Pandas จัดการ table ข้อมูลพร้อม label วันที่ ทั้งสองรวมกันทำให้วิเคราะห์ข้อมูลตลาดหลักหมื่นแถวได้ใน milliseconds

9.1 NumPy — คณิตศาสตร์บน Array

```
import numpy as np

# สร้าง array
prices = np.array([63000, 65000, 64500, 66000, 65800])

# Operations ทำทั้ง array พร้อมกัน (vectorized)
returns = np.diff(prices) / prices[:-1] # return ทุกวัน
print(returns)
# [0.0317 -0.0077  0.0233 -0.0030]

# Statistics
print(f"Mean:    {np.mean(returns):.4f}")
print(f"Std:     {np.std(returns):.4f}")
print(f"Max:      {np.max(returns):.4f}")
print(f"Min:      {np.min(returns):.4f}")

# Annualized Vol
annual_vol = np.std(returns) * np.sqrt(252) # ใช้ 252 เพื่อความง่าย — crypto เทรด 24/7 จริงๆ ใน
print(f"Annual Vol: {annual_vol:.1%}")
```

ทำไม NumPy เร็วกว่า Python loop?

Python loop: อ่านแต่ละ element → แปลง type → คำนวณ → เก็บ → ไปต่อ (overhead สูง)

NumPy: ส่ง array ทั้งก้อนให้ C code → คำนวณ parallel → ได้ผล (overhead ต่ำมาก)

ผลลัพธ์: array ขนาด 1,000,000 element → NumPy เร็วกว่า loop ประมาณ 100–1,000x

9.2 ทำไม "ตารางธรรมดา" ถึงไม่พอ

ก่อนตอบว่า DataFrame คืออะไร ลองดูก่อนว่าถ้าไม่มีมัน ชีวิตเป็นยังไง — เก็บราคา 2 เหรียญด้วยเครื่องมือที่เรามีจากบทที่ 8 (list ซ้อน list = "ตารางทำมือ"):

```
# ตารางธรรมดา: list ซ้อน list
table = [
    # [วันที่,      BTC,      ETH]
    ["2024-01-01", 63000, 3100],
    ["2024-01-02", 65000, 3250],
    ["2024-01-03", 64500, 3200],
]

# คำถามง่าย ๆ: "ราคา BTC วันที่ 2024-01-02 คือเท่าไร?"
# → ต้องเขียน loop ค้นเอง:
for row in table:
    if row[0] == "2024-01-02":      # ช่องที่ 0 คือวันที่ (ต้องจำเอง)
        print(row[1])              # ช่องที่ 1 คือ BTC (ต้องจำเอง)
```

ใช้งานได้ แต่มีปัญหา 3 ข้อที่จะกักรบกวนเราแน่นอนเมื่อข้อมูลโตขึ้น:

ปัญหาของตารางทำมือ

1. **ต้องจำตำแหน่งเอง** — "ช่องที่ 1 คือ BTC" ไม่มีเขียนไว้ในโค้ด ถ้าสลับคอลัมน์ในไฟล์ข้อมูลวันไหน โค้ดจะหิบลเลขผิดช่องโดยไม่มี error
2. **ค้นหาต้อง loop ทุกครั้ง** — ข้อมูล 100,000 แถว อยากรู้ราคาวันเดียว ต้องไล่ดูแสนแถว
3. **ข้อมูลหายแล้วแถวเลื่อน** — สมมติ ETH ไม่มีข้อมูลวันที่ 2 ถ้าเก็บแยกสอง list แล้วเอามาคู่กัน ตำแหน่งที่ 1 ของ BTC จะจับคู่กับ ETH ผิดวัน → คำนวณ correlation ผิดทั้งชุด แบบเงิบ ๆ

pandas ถูกสร้างมาแก้ 3 ปัญหานี้ตรง ๆ — และกุญแจของมันคือสิ่งที่เรียกว่า **index**

9.3 Index คืออะไร — ป้ายชื่อประจำแถว

list ธรรมดาเรียกข้อมูลด้วย "ตำแหน่ง": ตัวที่ 0, ตัวที่ 1, ตัวที่ 2 — เหมือนล็อกเกอร์ที่มีแต่เลขตัว

Series ของ pandas คือ list ที่ทุกค่ามีป้ายชื่อติดอยู่ — ป้ายชื่อเหล่านั้นรวมกันเรียกว่า index:

```
import pandas as pd

prices = pd.Series(
    [63000, 65000, 64500],          # ค่า (values)
    index=["2024-01-01", "2024-01-02", "2024-01-03"], # ป้ายชื่อ (index)
    name="BTC"
)

print(prices)
```

```
► OUTPUT 2024-01-01 63000 2024-01-02 65000 2024-01-03 64500 Name: BTC, dtype: int64
```

สังเกตว่า pandas พิมพ์ป้ายชื่อ (ซ้าย) คู่กับค่า (ขวา) เสมอ — ที่นี้คำถาม "ราคาวันที่ 2 ม.ค.?" ตอบได้บรรทัดเดียว ไม่ต้อง loop:

```
print(prices["2024-01-02"]) # 65000 — ถามด้วยป้ายชื่อตรง ๆ
print(prices.iloc[1])      # 65000 — หรือถามด้วยตำแหน่งแบบเดิมก็ทำได้ (iloc)
```

📌 **อ่านว่า:** หยิบค่าที่ป้ายชื่อ "2024-01-02" จาก prices — index ทำให้เราคู่กับข้อมูลด้วย "ภาษาของข้อมูล" (วันที่) แทนตำแหน่ง

และพลังที่สำคัญที่สุดของ index — **จับคู่แถวให้อัตโนมัติ (alignment)** ซึ่งแก้ปัญหาคำข้อ 3 ได้อย่างถาวร:

```
# ETH ไม่มีข้อมูลวันที่ 2 (ตลาดล่ม)
btc = pd.Series([63000, 65000, 64500],
                index=["2024-01-01", "2024-01-02", "2024-01-03"])
eth = pd.Series([3100, 3200],
                index=["2024-01-01", "2024-01-03"]) # ขาดวันที่ 2!

ratio = btc / eth # pandas จับคู่ด้วย "ป้ายชื่อ" ไม่ใช่ตำแหน่ง
print(ratio)
```

```
► OUTPUT 2024-01-01 20.322581 2024-01-02 NaN 2024-01-03 20.156250 dtype: float64
```

วันที่ 2 ได้ `NaN` (Not a Number = "ไม่มีข้อมูล") — pandas บอกตรง ๆ ว่าวันนี้คำนวณไม่ได้ แทนที่จะจับคู่ผิดแถวเจิบ ๆ แบบ list ถ้าเป็นตารางทำมือ ราคา ETH วันที่ 3 จะถูกจับคู่กับ BTC วันที่ 2 — ผิดโดยไม่มีใครรู้

9.4 DataFrame — หลาย Series ที่ใช้ป้ายชื่อชุดเดียวกัน

พอเข้าใจ Series แล้ว DataFrame ตรงไปตรงมามาก: **DataFrame = ตารางที่เกิดจากหลาย Series (หลายคอลัมน์) มาแชร์ index ชุดเดียวกัน** — หน้าตาเหมือน Excel sheet แต่อยู่ใน Python:

```
data = {
    "BTC": [63000, 65000, 64500, 66000, 65800],
    "ETH": [3100, 3250, 3200, 3300, 3280],
    "SOL": [140, 148, 145, 152, 149],
}
dates = pd.date_range("2024-01-01", periods=5) # สร้างป้ายวันที่ 5 วัน
df = pd.DataFrame(data, index=dates)

print(df)
```

```
► OUTPUT BTC ETH SOL 2024-01-01 63000 3100 140 2024-01-02 65000 3250 148 2024-01-03 64500
3200 145 2024-01-04 66000 3300 152 2024-01-05 65800 3280 149
```

กายวิภาคของ DataFrame มี 3 ส่วน — รู้ 3 ची่นี่แล้วอ่านเอกสาร pandas ออกทั้งหมด:

ส่วน	คืออะไร	ในตัวอย่างบน
index	ป้ายชื่อแถว (แนวตั้งซ้ายสุด)	วันที่ 2024-01-01 ถึง 05
columns	ป้ายชื่อคอลัมน์ (แนวนอนบนสุด)	BTC, ETH, SOL
values	ตัวข้อมูลข้างใน (NumPy array)	ตัวเลขราคาทั้งหมด

แล้วใช้ Excel แทนได้ไหม?

ได้ — สำหรับ "คู่มือข้อมูล" Excel ดีมาก แต่สำหรับงาน Quant มี 3 จุดที่ DataFrame ชนะขาด:

1. ทำซ้ำได้: โค้ด 10 บรรทัดรันกับข้อมูลใหม่ทุกวันอัตโนมัติ — Excel ต้องลากสูตรใหม่ทุกครั้ง

2. ขนาด: ข้อมูล 1 นาทีย้อนหลัง 3 ปี ≈ 1.5 ล้านแถว — Excel เริ่มค้าง pandas ทำงานเป็นวินาที

3. ตรวจสอบย้อนหลังได้: ทุกขั้นตอนคำนวณเขียนเป็นโค้ดอ่านได้ ส่งให้คนอื่น (หรือ AI) ตรวจสอบได้ — สูตรที่ฝังใน cell ของ Excel ตรวจสอบยากมาก

คิดง่าย ๆ: Excel = ทำกับข้าวด้วยมือ, DataFrame = โรงงานอาหาร — ทำกินเองมือเดียวใช้มือได้ แต่ถ้าต้องทำทุกวัน วันละหมื่นจาน ต้องใช้โรงงาน

"แล้วมันทำอะไรได้บ้าง?" — ตัวอย่างงานที่ใช้บ่อยที่สุดใน Quant แต่ละงานใช้โค้ดบรรทัดเดียว (ทุกอันได้ผลถูกต้องแม้ข้อมูลขาดบางวัน เพราะ index จับคู่ให้):

อยากได้	โค้ด
return รายวันของทุกเหรียญ	<code>df.pct_change()</code>
ค่าเฉลี่ยเคลื่อนที่ 20 วันของ BTC	<code>df["BTC"].rolling(20).mean()</code>
เฉพาะวันที่ BTC ตกเกิน 2%	<code>df[df["BTC"].pct_change() < -0.02]</code>
correlation ระหว่างทุกคู่เหรียญ	<code>df.pct_change().corr()</code>
กราฟราคาทุกเหรียญ	<code>df.plot()</code>

9.5 Operations ที่ใช้บ่อยใน Quant

```
# --- ดูข้อมูลเบื้องต้น ---
df.head(3)          # 3 แถวแรก
df.tail(3)          # 3 แถวสุดท้าย
df.describe()       # mean, std, min, max, quartile
df.shape            # (5, 3) = 5 แถว 3 คอลัมน์
df.dtypes           # type ของแต่ละคอลัมน์

# --- เลือกข้อมูล ---
df["BTC"]           # คอลัมน์ BTC (Series)
df[["BTC", "ETH"]]  # สองคอลัมน์ (DataFrame)
df.loc["2024-01-03"] # .loc: เลือกด้วย "ป้ายชื่อ" (label) = วันที่
df.iloc[0]          # .iloc: เลือกด้วย "ตำแหน่ง" (position) = แถวที่ 0 คือแถวแรก
df.iloc[-1]         # .iloc[-1] = แถวสุดท้าย (เลขลบนับจากท้าย เหมือน list)

# --- คำนวณ Return ---
returns = df.pct_change() # daily return ทุก column (แถวแรกเป็น NaN เพราะไม่มีวันก่อน)
log_returns = np.log(df/df.shift(1)) # log return: shift(1) เลื่อนข้อมูลลงหนึ่งแถว (= ราคาเมื่อวาน)

# --- Rolling Statistics ---
rolling_mean = df["BTC"].rolling(window=3).mean() # ค่าเฉลี่ย 3 วันล่าสุด (2 แถวแรกเป็น NaN)
rolling_std = df["BTC"].rolling(window=3).std() # std 3 วันล่าสุด
zscore = (df["BTC"] - rolling_mean) / rolling_std # z-score: ห่างจากค่าเฉลี่ยกี่ std
```

.loc vs .iloc — เลือกตัวไหน?

	<code>.loc["2024-01-03"]</code>	<code>.iloc[2]</code>
เลือกด้วย	ป้ายชื่อ (วันที่, ชื่อหุ้น)	ตำแหน่ง (0, 1, 2, -1)
ข้อดี	ชัดเจน ไม่ผิดพลาดแม้เรียงลำดับใหม่	เร็ว ใช้ได้แม้ไม่รู้ชื่อ index
ใช้เมื่อ	รู้วันที่/label ที่ต้องการ	ต้องการ "ตัวแรก" หรือ "ตัวสุดท้าย"

NaN คืออะไร? "Not a Number" = "ไม่มีข้อมูล ณ จุดนี้" — pandas ใส่อัตโนมัติเมื่อคำนวณไม่ได้ (เช่น แถวแรกของ `pct_change()` หรือหน้าตาแรกของ `rolling()`) — `dropna()` ลบแถวที่มี NaN ออก, `fillna(0)` แทน NaN ด้วยศูนย์ — pandas ส่วนใหญ่ข้าม NaN ให้อัตโนมัติ (`mean()` , `std()`) แต่บางงาน (เช่น `corr()`) ต้องการ `dropna()` ก่อน

```
# --- Filter: boolean indexing — "กรองแถวด้วยเงื่อนไข" ---
btc_returns = df["BTC"].pct_change()

# แทนที่จะ loop แล้ว if เองทีละแถว pandas ทำให้ในบรรทัดเดียว:
crash_days = df[btc_returns < -0.02] # เลือกเฉพาะแถวที่ return < -2%
print(f"BTC crash days (<-2%): {len(crash_days)}")
```

```
# เงื่อนไขสองข้อ: ต้องใช้ & และ กรอบแต่ละเงื่อนไขด้วย ()
both_up = df[(btc_returns > 0) & (df["ETH"].pct_change() > 0)]
```

ไฟล์ btc_daily.csv มาจากไหน?

มีสองทาง — ทางที่ 1 (แนะนำสำหรับตอนนี้: ไม่ต้องดาวน์โหลดอะไรเลย รันบล็อกด้านล่างเพื่อสร้างไฟล์จำลองขึ้นมาเอง แล้วโค้ดทั้งหัวข้อนี้อันจะรันได้ทันที

ทางที่ 2: ใช้ข้อมูลจริง — `pip install yfinance` แล้ว

```
import yfinance as yf
raw = yf.download("BTC-USD", start="2023-01-01", auto_adjust=True)
raw.to_csv("btc_daily.csv")
```

⚠ แต่ไฟล์ที่ได้จะยังใช้กับโค้ดด้านล่างไม่ได้ทันที เพราะแหล่งข้อมูลจริงตั้งชื่อคอลัมน์เป็นตัวใหญ่ (Close ไม่ใช่ close) และตั้งชื่อ index ว่า Date — ถ้าเจอ `KeyError: 'close'` คือเรื่องนี้ไม่ใช่คุณพิมพ์ผิด นี่คือนักพัฒนาของงานข้อมูลจริง จึงต้องมีขั้น "จัดบ้าน" ก่อนใช้เสมอ:

```
df = pd.read_csv("btc_daily.csv", index_col=0, parse_dates=True)

# บางเวอร์ชันของ yfinance ให้หัวตารางสองชั้น (MultiIndex) — ยับให้เหลือชั้นเดียว
if isinstance(df.columns, pd.MultiIndex):
    df.columns = df.columns.get_level_values(0)

df.columns = df.columns.str.lower() # Close -> close
df.index.name = "date"             # Date -> date
print(df.columns.tolist())         # เช็คก่อนใช้เสมอ
```

🔗 **นิสัยที่ควรติดตัว:** เปิดไฟล์ข้อมูลใหม่ทุกครั้งให้ `print(df.columns.tolist())` และ `df.head()` ก่อนเขียนโค้ดต่อ — ประหยัดเวลาดี๊ดีักได้มหาศาล

Running Example: โหลดข้อมูลจาก CSV และคำนวณ Metrics

```
# — สร้างไฟล์จำลองก่อน เพื่อให้รันได้ทันทีโดยไม่ต้องดาวน์โหลด —
# (ถ้ามี btc_daily.csv จริงแล้ว ข้าม 8 บรรทัดนี้ไปได้เลย)
np.random.seed(42)
dates_sim = pd.date_range("2023-01-01", periods=300, freq="D")
close_sim = 30000 * np.exp(np.cumsum(np.random.normal(0.001, 0.02, 300)))
pd.DataFrame({
    "date": dates_sim, "open": close_sim * 0.999, "high": close_sim * 1.01,
    "low": close_sim * 0.99, "close": close_sim,
```



```

    "volume": np.random.uniform(1e4, 5e4, 300),
}).to_csv("btc_daily.csv", index=False)

# — ตั้งแต่บรรทัดนี้คือโค้ดที่ใช้กับไฟล์จริงได้เหมือนกัน —
# ไฟล์ btc_daily.csv มีคอลัมน์ date, open, high, low, close, volume
df = pd.read_csv("btc_daily.csv", index_col="date", parse_dates=True)
df = df.sort_index()          # เรียงวันที่จากเก่าไปใหม่

close = df["close"]          # หยิบคอลัมน์ราคาปิดมาตั้งชื่อสั้น ๆ

# 1) return รายวัน
df["return"] = close.pct_change() # เปอร์เซ็นต์เปลี่ยนแปลงจากวันก่อน

# 2) log return
prev_close = close.shift(1)      # ราคาเมื่อวาน (เลื่อนลง 1 แถว)
df["log_ret"] = np.log(close / prev_close)

# 3) annualized volatility จากหน้าต่าง 20 วัน
daily_vol_20 = df["return"].rolling(20).std() # std ของ return 20 วันล่าสุด
df["vol_20"] = daily_vol_20 * np.sqrt(252)    # ใช้ 252 เพื่อความง่าย — crypto เทรด 24

# 4) moving average 50 วัน
df["ma_50"] = close.rolling(50).mean()

# 5) z-score: ราคาวันนี้ห่างค่าเฉลี่ย 60 วัน กี่ std
mean_60 = close.rolling(60).mean() # ค่าเฉลี่ยเคลื่อนที่ 60 วัน
std_60 = close.rolling(60).std()   # std เคลื่อนที่ 60 วัน
df["zscore"] = (close - mean_60) / std_60 # สูตร z-score เดิมจากบทที่ 7

print(df.tail()) # ดูผลลัพธ์ 5 วันล่าสุด
print(f"\nAnnualized Vol (20d): {df['vol_20'].iloc[-1]:.1%}")
print(f"Current Z-score: {df['zscore'].iloc[-1]:.2f}")

```

AI จะยุบ 5 ขั้นนี้เหลือแบบนี้ — ฝึกอ่าน method chain

```
df["vol_20"] = df["close"].pct_change().rolling(20).std() * np.sqrt(252)
```

เทคนิคอ่าน chain: อ่านจากซ้ายไปขวา เหมือนสายพานโรงงาน — ข้อมูลผ่านเครื่องจักรทีละตัว จุดคือ "ส่งต่อให้"

1. `df["close"]` — เริ่มจากราคาปิด
2. `.pct_change()` — ส่งเข้าเครื่องคำนวณ return รายวัน
3. `.rolling(20)` — ส่งต่อเข้าหน้าต่างเลื่อน 20 วัน
4. `.std()` — คำนวณ std ของแต่ละหน้าต่าง

5. * np.sqrt(252) — สุดท้ายคุณ $\sqrt{252}$ แปลงเป็นรายปี

คือขั้นที่ 1 + 3 ของเรารวมกันในบรรทัดเดียว — chain สั้น ๆ (2–3 จุด) อ่านสบาย แต่ถ้ายาวกว่านั้นแล้วเริ่มงง แดกเป็น
ตัวแปรกลางแบบที่เราเขียนได้เสมอ ผลเหมือนกันปะ และ debug ง่ายกว่าเพราะ print ดูค่ากลางทางได้

► แบบฝึกหัด: คำนวณ Correlation Matrix

บทที่ 10: Visualization

แก่นของบทนี้

กราฟที่ดีบอกเรื่องราวได้ภายใน 3 วินาที — ใน Quant ใช้กราฟเพื่อ debug signal, ตรวจสอบ backtest, และนำเสนอผลลัพธ์ บทนี้ใช้ matplotlib เป็นหลัก (standard, ทำได้ทุกอย่าง) และแนะนำ plotly สำหรับ interactive charts

10.1 matplotlib พื้นฐาน

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# สร้างข้อมูลตัวอย่าง
dates = pd.date_range("2024-01-01", periods=60)
prices = 65000 * np.cumprod(1 + np.random.normal(0.001, 0.02, 60))

# Plot พื้นฐาน
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(dates, prices, color="#0d9488", linewidth=1.5, label="BTC Price")
ax.set_title("BTC/USD Daily Close", fontsize=14, fontweight="bold")
ax.set_xlabel("Date")
ax.set_ylabel("Price (USD)")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("btc_price.png", dpi=150)
plt.show()
```

10.2 Multiple Subplots — Price + Return + Z-score

```
# 3 panels: ราคา, return, z-score
fig, axes = plt.subplots(3, 1, figsize=(10, 10), sharex=True)

# สมมติ df มีคอลัมน์ close, return, zscore

# Panel 1: ราคา + MA50
axes[0].plot(df.index, df["close"], label="Close", color="#0d9488")
axes[0].plot(df.index, df["ma_50"], label="MA50", color="#f59e0b", linestyle="--")
```

```

axes[0].set_ylabel("Price ($)")
axes[0].legend()
axes[0].set_title("BTC Analysis Dashboard")

# Panel 2: Daily Return — เลือกสีแท่งทีละวัน: เขียวถ้าบวก แดงถ้าลบ
bar_colors = []
for r in df["return"]:
    if r > 0:
        bar_colors.append("#16a34a")    # เขียว
    else:
        bar_colors.append("#dc2626")    # แดง

axes[1].bar(df.index, df["return"], color=bar_colors, alpha=0.7, width=0.8)
axes[1].axhline(0, color="black", linewidth=0.5)
axes[1].set_ylabel("Daily Return")

# Panel 3: Z-score + threshold lines
axes[2].plot(df.index, df["zscore"], color="#7c3aed", linewidth=1)
axes[2].axhline(2.0, color="#dc2626", linestyle="--", alpha=0.7, label="Entry ( $\pm 2\sigma$ )")
axes[2].axhline(-2.0, color="#dc2626", linestyle="--", alpha=0.7)
axes[2].axhline(0, color="black", linewidth=0.5)
axes[2].set_ylabel("Z-score")
axes[2].legend()

plt.tight_layout()
plt.savefig("btc_dashboard.png", dpi=150)
plt.show()

```

🤖 AI จะยัด loop เลือกสีเข้าไปในวงเล็บเลย — ฟีกอ่าน

```

axes[1].bar(df.index, df["return"],
            color=["#16a34a" if r > 0 else "#dc2626" for r in df["return"]])

```

ตรง `color=[...]` คือ comprehension ที่มีเงื่อนไขเลือกค่า (A if เงื่อนไข else B) ฝังอยู่:

1. `for r in df["return"]` — หยิบ return ทีละวัน (loop เดิม)
2. `"#16a34a" if r > 0 else "#dc2626"` — อ่านว่า "เอาสีเขียว ถ้า r บวก ไม่งั้นเอาสีแดง" (รูปแบบนี้เรียก ternary — ค่าที่ได้อยู่หน้า if)
3. ผลคือ list ของสี ส่งให้ `color=` โดยตรง ไม่ต้องตั้งชื่อตัวแปร

ความหมายเหมือน loop 6 บรรทัดของเรา — ถ้าเจอแบบนี้ในโค้ด AI แล้วงง ให้ลากมันออกมาตั้งเป็นตัวแปรข้างนอกก่อน แล้วค่อยอ่าน

10.3 Scatter Plot — Correlation

```

# สร้าง DataFrame ที่มีราคา BTC และ ETH (จำลองข้อมูล 100 วัน)
import numpy as np, pandas as pd
np.random.seed(42)
dates = pd.date_range("2024-01-01", periods=100)
df_multi = pd.DataFrame({
    "BTC": 65000 * np.cumprod(1 + np.random.normal(0.001, 0.02, 100)),
    "ETH": 3200 * np.cumprod(1 + np.random.normal(0.001, 0.025, 100)),
}, index=dates)

# วิเคราะห์ correlation ระหว่าง BTC และ ETH returns
btc_ret = df_multi["BTC"].pct_change().dropna() # return รายวัน ลบแถวแรกที่เป็น NaN
eth_ret = df_multi["ETH"].pct_change().dropna()

fig, ax = plt.subplots(figsize=(6, 6))
ax.scatter(btc_ret, eth_ret, alpha=0.4, color="#0d9488", s=20)

# เพิ่ม regression line
m, b = np.polyfit(btc_ret, eth_ret, 1) # slope, intercept
x_line = np.linspace(btc_ret.min(), btc_ret.max(), 100)
ax.plot(x_line, m * x_line + b, color="#dc2626", linewidth=2,
        label=f" $\beta = {m:.2f}$ ")

ax.set_xlabel("BTC Daily Return")
ax.set_ylabel("ETH Daily Return")
ax.set_title(f"BTC vs ETH Returns (r={btc_ret.corr(eth_ret):.2f})")
ax.legend()
plt.tight_layout()
plt.show()

```

Matplotlib Cheat Sheet — สำหรับ Quant

Function	ใช้ทำ
<code>ax.plot(x, y)</code>	เส้นราคา, equity curve
<code>ax.bar(x, y)</code>	return รายวัน, volume
<code>ax.scatter(x, y)</code>	correlation, scatter plot
<code>ax.fill_between(x, y1, y2)</code>	Bollinger Bands, confidence interval
<code>ax.axhline(y)</code>	เส้น threshold (entry/exit level)
<code>ax.axvspan(x1, x2)</code>	ไฮไลต์ช่วงเวลา (crisis, drawdown)

ตัวอย่าง: Equity Curve + Drawdown

```
import matplotlib.pyplot as plt
import numpy as np

# จำลอง equity curve
np.random.seed(42)
daily_ret = np.random.normal(0.0005, 0.015, 252)
equity = 100000 * np.cumprod(1 + daily_ret)

# คำนวณ drawdown
running_max = np.maximum.accumulate(equity)
drawdown = (equity - running_max) / running_max

# Plot
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 6), sharex=True)

ax1.plot(equity, color="#0d9488", linewidth=1.5)
ax1.set_ylabel("Portfolio Value ($)")
ax1.set_title("Strategy Equity Curve")
ax1.grid(True, alpha=0.3)

ax2.fill_between(range(len(drawdown)), drawdown, 0,
                 color="#dc2626", alpha=0.5)
ax2.set_ylabel("Drawdown")
ax2.set_xlabel("Trading Day")
ax2.grid(True, alpha=0.3)

max_dd = drawdown.min()
final = equity[-1]
print(f"Final Equity:   ${final:,.0f}")
print(f"Total Return:   {(final/100000-1):.1%}")
print(f"Max Drawdown:   {max_dd:.1%}")

plt.tight_layout()
plt.show()
```

ทำไม่ชื่อกราฟในเล่มนี้เป็นภาษาอังกฤษทั้งหมด

ถ้าใส่ข้อความไทยใน `set_title()` / `set_xlabel()` / `label=` ตรงๆ matplotlib จะพิมพ์ออกมาเป็น **สี่เหลี่ยมเปล่า** `□□□` พร้อมคำเตือน `Glyph missing from font(s) DejaVu Sans` — เพราะฟอนต์ติดตั้งของ matplotlib ไม่มีตัวอักษรไทย

แก้ได้ถ้าต้องการไทยจริงๆ (ต้องมีไฟล์ฟอนต์ในเครื่อง):

```
import matplotlib
matplotlib.rcParams["font.family"] = "Sarabun" # หรือ "TH Sarabun New", "Noto Sans
matplotlib.rcParams["axes.unicode_minus"] = False # กันเครื่องหมายลบกลายเป็นสี่เหลี่ยม
```

เล่มนี้เลือกใช้**อังกฤษในกราฟไทยในคำอธิบาย** เพื่อให้โค้ดทุกบล็อกกันได้เหมือนกันทุกเครื่องโดยไม่ต้องติดตั้งฟอนต์ก่อน

10.4 Histogram + Bell Curve — รูปร่างของการกระจาย

สามหัวข้อที่ผ่านมาดู "ราคาเดินทางไปทางไหน" (เวลาอยู่แกน x) · หัวข้อนี้เปลี่ยนคำถามเป็น "ผลตอบแทนกระจายตัวอย่างไร" — ทิ้งเวลาไปเลย เอาแต่ค่ามากองรวมกันแล้วดูรูปร่าง

แก่นของหัวข้อนี้

Histogram ตอบคำถามที่กราฟเส้นตอบไม่ได้: "วันที่เหวี่ยงแรง ๆ เกิดบ่อยแค่ไหน?" · และเมื่อวาดเส้นระฆัง (Normal) ทับลงไป คุณจะเห็นด้วยตาว่าผลตอบแทนจริงไม่ใช่ระฆัง — หางหนากว่ามาก · นี่คือสาเหตุที่โมเดลความเสี่ยงซึ่งสมมติ Normal ประเมินความเสี่ยงต่ำเกินไปเสมอ

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

# จำลอง return รายวัน 3 ปี แบบที่คล้ายของจริง:
# ส่วนใหญ่เป็นวันธรรมดา + 3% ของวันเป็นวันข่าวแรง (jump)
np.random.seed(3)
n = 750
btc_ret = pd.Series(
    np.random.normal(0.0005, 0.018, n)
    + np.random.choice([0, 1], n, p=[0.97, 0.03]) * np.random.normal(0, 0.07, n),
    name="BTC")
eth_ret = pd.Series(
    np.random.normal(0.0004, 0.026, n)
    + np.random.choice([0, 1], n, p=[0.96, 0.04]) * np.random.normal(0, 0.09, n),
    name="ETH")

mu, sd = btc_ret.mean(), btc_ret.std()

fig, ax = plt.subplots(figsize=(10, 5))

# ① Histogram — density=True เพื่อให้สเกลเทียบกับเส้นโค้งได้
ax.hist(btc_ret, bins=60, density=True, alpha=0.55,
```

```

        color="#0d9488", edgecolor="white", label="BTC returns (actual)")

# ② เส้นระฆัง Normal ที่มี mean/std เดียวกันปะ
x = np.linspace(btc_ret.min(), btc_ret.max(), 400)
ax.plot(x, stats.norm.pdf(x, mu, sd), color="#dc2626", lw=2.5,
        label=f"Normal( $\mu$ = $\{\mu:.4f\}$ ,  $\sigma$ = $\{\sigma:.4f\}$ )")

# ③ ขีดเส้น  $\pm 3\sigma$  ให้เห็นว่า "ขอบของโลก Normal" อยู่ไหน
for k in (-3, 3):
    ax.axvline(mu + k*sd, color="#7c3aed", ls="--", lw=1.2)
ax.text(mu + 3*sd, ax.get_ylim()[1]*0.7, " +3 $\sigma$ ", color="#7c3aed", fontsize=9)

ax.set_title("BTC Daily Returns vs Normal (look at both tails)")
ax.set_xlabel("Daily Return")
ax.set_ylabel("Density")
ax.legend()
ax.grid(True, alpha=0.3)

# ④ ตัวเลขที่ยืนยันสิ่งที่ตาเห็น
print(f"Skewness: {stats.skew(btc_ret):+.2f}      "
      f"Excess kurtosis: {stats.kurtosis(btc_ret):+.2f}      (Normal = 0.00, 0.00)")
print("\nวันที่เหวี่ยงเกิน k เท่าของ  $\sigma$  – จริง vs ที่ Normal ทำนาย")
for k in (1, 2, 3, 4, 5):
    actual = int((btc_ret.abs() > k*sd).sum())
    expected = 2 * stats.norm.sf(k) * n
    ratio = f"{actual/expected:>7.1f}x" if expected >= 0.05 else " >100x"
    print(f" |r| > {k} $\sigma$  : {actual:4d} วัน · Normal {expected:7.1f} วัน · {ratio}")

plt.tight_layout()
plt.show()

```

► OUTPUT Skewness: +0.55 Excess kurtosis: +10.89 (Normal = 0.00, 0.00) วันที่เหวี่ยงเกิน k เท่าของ σ – จริง vs ที่ Normal ทำนาย |r| > 1σ : 158 วัน · Normal 238.0 วัน · 0.7x |r| > 2σ : 24 วัน · Normal 34.1 วัน · 0.7x |r| > 3σ : 10 วัน · Normal 2.0 วัน · 4.9x |r| > 4σ : 7 วัน · Normal 0.0 วัน · >100x |r| > 5σ : 4 วัน · Normal 0.0 วัน · >100x

🔴**อ่านว่า:** density=True คือกฎของบอล็อกนี้ — ปกติ hist นับ "จำนวนวัน" ในแต่ละแท่ง แต่ density=True เปลี่ยนเป็นสัดส่วนที่ทำให้พื้นที่รวมทั้งกราฟ = 1 · จำเป็นเพราะ norm.pdf ก็มีพื้นที่รวม 1 เหมือนกัน — ถ้าไม่ใส่เส้นแดงจะแบนติดพื้นจนมองไม่เห็นอะไรเลย · สองสิ่งจะเทียบกันได้ ต้องอยู่สเกลเดียวกันก่อน

อ่านตารางนี้ให้ออก — นี่คือ "fat tail" ที่ทุกคนพูดถึง

สังเกตว่ามันแปลกสองทางพร้อมกัน:

- ช่วง 1σ – 2σ เกิดน้อยกว่า Normal (0.7 เท่า) — วันที่ "เหวี่ยงพอประมาณ" มีน้อยกว่าที่ทฤษฎีบอก

- ช่วงเกิน 3σ เกิดมากกว่ามหาศาล — 3σ เกิด 4.9 เท่า · 4σ กับ 5σ เกินร้อยเท่า (Normal ทำนายว่าแทบไม่ควรถูกเกิดเลยใน 750 วัน)

แปลว่าผลตอบแทนจริงไม่ได้กระจายกว้างกว่า Normal ทั้งกระดาน — มันเอามวลจากช่วงกลาง ๆ ไปกองไว้ที่ *ตรงกลางสุด* (วันเจียบ) และ *ปลายหาง* (วันหายนะ) · ตลาดจึงดู "นิ่งกว่าที่คิด" เป็นเดือน ๆ แล้วขยับทีเดียวนพอร์ทพัง

🔴 **ผลทางปฏิบัติ:** โมเดลที่สมมติ Normal จะบอกว่าการเหวี่ยง -4σ "เกิดทุก 40 ปี" แต่ตารางข้างบนบอกว่ามันเกิด **7 ครั้งใน 3 ปี** · ทุกครั้งที่เห็นคำว่า "เหตุการณ์ 1 ในล้านปี" ในรายงานความเสี่ยงให้ถามก่อนว่าเขาสมมติการกระจายแบบไหน

10.5 Box Plot — เปรียบการกระจายหลายสินทรัพย์

Histogram ดูได้ที่ละตัว · เปรียบหลายตัวพร้อมกันใช้ **box plot** ซึ่งเปรียบเทียบการกระจายทั้งก้อนให้เหลือกล่องเดียว

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.5))

# — ซ้าย: box plot เปรียบ BTC กับ ETH —
#หมายเหตุ: ไม่ใส่ labels= ใน boxplot() เพราะ matplotlib รุ่นใหม่
# เปลี่ยนชื่อเป็น tick_labels — ตั้งชื่อแกนแยกจึงเข้ากันได้ทุกเวอร์ชัน
bp = ax1.boxplot([btc_ret, eth_ret],
                  patch_artist=True, showfliers=True,
                  medianprops=dict(color="#111827", lw=2))
for patch, color in zip(bp["boxes"], ["#0d9488", "#7c3aed"]):
    patch.set_facecolor(color)
    patch.set_alpha(0.45)

ax1.set_xticks([1, 2])
ax1.set_xticklabels(["BTC", "ETH"])
ax1.axhline(0, color="#9ca3af", lw=1)
ax1.set_title("Daily Return Distribution")
ax1.set_ylabel("Daily Return")
ax1.grid(True, alpha=0.3, axis="y")

# — ขวา: histogram ซ้อนกัน เพื่อเทียบกับ box plot —
ax2.hist(btc_ret, bins=50, alpha=0.5, density=True,
         color="#0d9488", label="BTC")
ax2.hist(eth_ret, bins=50, alpha=0.5, density=True,
         color="#7c3aed", label="ETH")
ax2.set_title("Same data as histogram")
ax2.set_xlabel("Daily Return")
ax2.legend()
ax2.grid(True, alpha=0.3)

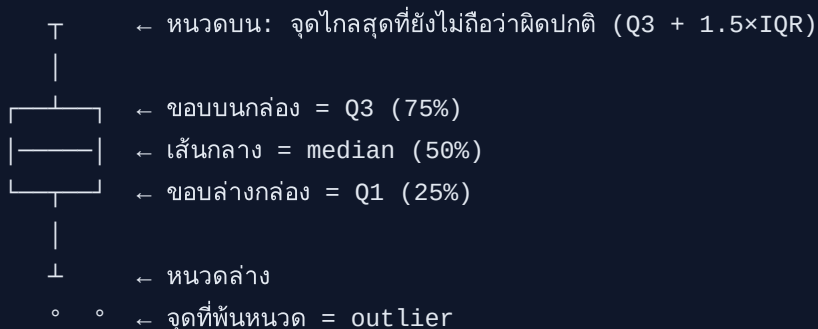
# ตัวเลขที่ box plot กำลังวาด
```

```
for s in (btc_ret, eth_ret):
    q1, med, q3 = s.quantile([0.25, 0.5, 0.75])
    iqr = q3 - q1
    n_out = int(((s < q1 - 1.5*iqr) | (s > q3 + 1.5*iqr)).sum())
    print(f"{s.name}: median={med:+.4f} IQR={iqr:.4f} "
          f"outlier={n_out} วัน ({n_out/len(s):.1%})")

plt.tight_layout()
plt.show()
```

► OUTPUT BTC: median=+0.0008 IQR=0.0255 outlier=17 วัน (2.3%) ETH: median=+0.0009 IQR=0.0321 outlier=20 วัน (2.7%)

📊 box plot บอกอะไร — 5 ตัวเลขในกล่องเดียว



1. กล่อง = ช่วงที่ข้อมูล 50% กลาง อยู่ · ความสูงของกล่องคือ $IQR = Q3 - Q1$ · BTC IQR 0.0255 vs ETH 0.0321 → ETH เหวี่ยงกว่าในวันธรรมดา ~26%

2. เส้นกลาง = median ไม่ใช่ mean — outlier ดึงไม่ได้ จึงบอก "วันปกติ" ได้ตรงกว่าค่าเฉลี่ย

3. หนด ยาวถึง $1.5 \times IQR$ จากขอบกล่อง (เกณฑ์มาตรฐาน ไม่ใช่กฎธรรมชาติ)

4. จุดที่พ้นหนด = สิ่งที่ box plot เรียกว่า outlier — BTC มี 44 วัน (5.9%) · ⚠️ ในโลกการเงินอย่าเรียกมันว่า "ข้อมูลผิดพลาด" เพราะวันเหล่านั้นคือวันที่กำไรและขาดทุนจริงเกิดขึ้น

เทียบสองภาพซ้าย-ขวา: box plot บอก "ตรงกลางอยู่ไหน กว้างแคไหน" ได้เร็วและเทียบหลายตัวได้ · histogram บอก "รูปร่างเป็นอย่างไร มีสองยอดไหม เบื้องไหน" ซึ่ง box plot มองไม่เห็น — ใช้คู่กัน ไม่ใช่เลือกอย่างเดียว

🔵 [รอบสอง] histogram หลอกตาได้ และเครื่องมือที่แม่นยำกว่าสำหรับดูหาง

จำนวน bins เปลี่ยนข้อสรุปได้ — ข้อมูลชุดเดิมปะะ ๆ ถ้าใช้ `bins=10` จะดูเป็นระฆังสวย ๆ แต่ `bins=200` จะดูขาดวันเป็นหนาม · ไม่มีค่าที่ "ถูก" · ลองสองสามค่าเสมอก่อนสรุปอะไรจากรูปร่าง (หรือใช้ `bins="fd"` ให้ matplotlib เลือกด้วยกฎ Freedman-Diaconis)

```
# ดูผลของ bins ต่อข้อสรุป — ข้อมูลชุดเดียวกันทั้งสามภาพ
fig, axes = plt.subplots(1, 3, figsize=(12, 3.2))
for ax, b in zip(axes, [10, 60, 200]):
```

```
ax.hist(btc_ret, bins=b, density=True, color="#0d9488", alpha=0.6)
ax.set_title(f"bins={b}")
ax.set_xlim(-0.15, 0.15)
plt.tight_layout(); plt.show()
```

สำหรับดูทางโดยเฉพาะ histogram เป็นเครื่องมือที่แย เพราะทางมีข้อมูลน้อย แท่งจึงเตี้ยจนมองไม่ออกว่าต่างจาก Normal แค่ไหน เครื่องมือที่ตรงงานคือ **QQ-plot** ซึ่งเอา quantile ของข้อมูลจริงไปพล็อตกับ quantile ของ Normal — ถ้าตรงกันจะได้เส้นตรง 45° ส่วนที่ *โก่งขึ้นปลายทั้งสองข้าง* คือทางที่หนากว่า Normal:

```
fig, ax = plt.subplots(figsize=(5.5, 5))
stats.probplot(btc_ret, dist="norm", plot=ax)
ax.set_title("QQ-plot: bends at both ends = fat tail")
ax.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()
```

📖 **โยงไปข้างหน้า:** ตัวเลข excess kurtosis +10.89 ที่เห็นในหัวข้อนี้จะกลับมาเป็นตัวละครหลักใน บทที่ 11 ตอนคำนวณ VaR — ที่นั่นเราจะเห็นว่าใช้ Normal คำนวณ VaR ทำให้ประเมินความเสี่ยงต่ำไปประมาณ 1.4 เท่า และทำไมต้องเปลี่ยนไปใช้ Student-t

10.6 plotly — Interactive Charts (แนะนำ)

```
# pip install plotly
import plotly.graph_objects as go

fig = go.Figure()

# Candlestick chart
fig.add_trace(go.Candlestick(
    x=df.index,
    open=df["open"],
    high=df["high"],
    low=df["low"],
    close=df["close"],
    name="BTC/USDT"
))

# เพิ่ม MA
fig.add_trace(go.Scatter(
    x=df.index, y=df["ma_50"],
    line=dict(color="#f59e0b", width=1.5),
    name="MA50"
))
```

```
fig.update_layout(
    title="BTC/USDT Candlestick",
    yaxis_title="Price (USD)",
    xaxis_rangeslider_visible=False,
    template="plotly_dark"
)
fig.show() # เปิดใน browser — zoom, pan ได้
```

► แบบฝึกหัด: Plot Spread Z-score พร้อม Entry/Exit Signals

จบ Part I — ทักษะที่ได้

ตอนนี้คุณสามารถ:

- อ่านโค้ดออกเสียงเป็นภาษาไทยได้ที่ละบรรทัด — รวมถึง comprehension และ method chain ที่ AI ชอบเขียน ✓
- เขียน Python function, loop, และ conditional logic สำหรับ signal generation ✓
- เก็บข้อมูลด้วย list, dict, tuple, set เลือกถูกตามสถานการณ์ ✓
- โหลดข้อมูลตลาดเข้า pandas DataFrame คำนวณ return, rolling vol, z-score ✓
- plot equity curve, drawdown, scatter plot, candlestick ด้วย matplotlib/plotly ✓

ใน **Part II** เราจะนำ Python ที่รู้มาต่อเข้ากับ math จาก Part 0: OLS regression, optimization, linear programming, cointegration

⚠ ก่อนไป Part II — สิ่งสำคัญที่ต้องจำ

1. **Lookahead bias:** signal ที่คำนวณจากข้อมูลวัน t ต้องถูก `.shift(1)` ก่อน execute วัน $t+1$ — Part IV จะอธิบายละเอียด แต่จำหลักนี้ไว้ตั้งแต่ตอนนี้
2. **Code ใน Part I ยังไม่ optimize:** loop แบบ manual ที่เขียนเร็วพอสำหรับเรียน แต่ช้ากว่า pandas vectorized ใน Part II ประมาณ 10–100x สำหรับข้อมูลหลายหมื่นแถว
3. ตัวเลข "ดูดี" ยังไม่น่าเชื่อ: Sharpe หรือ Return ที่คำนวณในตัวอย่าง Part I–II เป็น in-sample ทั้งหมด Part IV จะสอน Walk-Forward เพื่อยืนยันว่าดีจริงหรือ overfit

